

# Backend REST API gyakorló feladat

A feladat egy olyan REST API szolgáltatás létrehozása, amely pizzarendelések tételeit kezeli, és üzletileg értelmezhető lekérdezéseket, statisztikákat, valamint kimutatásokat készít.

Node.js esetén a szolgáltatásnak a **3000-es porton** kell elérhetőnek lennie.

Az API válaszok minden esetben **JSON** formátumúak legyenek.

## Adatforrás és architektúra

### 1. Adatbázis létrehozása

Hozzon létre adatbázist **pizzashop** néven a mellékelt CSV fájl felhasználásával.

CSV fájl neve: `pizza_rendeles_tetelek.csv`

Tábla neve: `order_items`

Mezők:

| Mező    | Jelentés                                                                    | Típus  |
|---------|-----------------------------------------------------------------------------|--------|
| id      | Egyedi azonosító, elsődleges kulcs                                          | int    |
| orderId | A rendelés azonosítója 1-22 között, egy rendeléshez több tétel is tartozhat | int    |
| name    | A megrendelt pizza neve                                                     | string |
| amount  | Az adott pizzából rendelt darabszám                                         | int    |
| price   | A pizza egységára forintban                                                 | int    |

**Megjegyzés:** A CSV **60 tételt** tartalmaz, az `orderId` mező **1 és 22** között mozog, és szándékosan ismétlődik, mert egy rendelés több tételből állhat.

### Opcionális SQL tábla definíció

```
CREATE TABLE order_items (  
  id INT PRIMARY KEY,  
  orderId INT NOT NULL,  
  rendeles VARCHAR(100) NOT NULL,  
  mennyiseg INT NOT NULL,  
  egysegar INT NOT NULL  
);
```

---

## Végpontok megvalósítása

---

Hozza létre az alábbi REST végpontokat a megadott útvonalakon és logikával.

---

### 1. SELECT / WHERE / ORDER BY feladatok

---

#### 1.1 Rendelési tételek lekérése rendelésazonosító alapján

GET `/api/order-items/order/{orderId}`

Leírás:

Adja vissza a megadott `orderId` -hoz tartozó összes tételt **id szerint növekvő sorrendben**.

Sikeres válasz (200 OK):

```
[  
  {  
    "id": 12,  
    "orderId": 4,  
    "rendeles": "BBQ csirkés",  
    "mennyiseg": 2,  
    "egysegar": 2990  
  }  
]
```

Hibakezelés:

- **404 Not Found:** ha a rendelésazonosítóhoz nem tartozik tétel

```
{ "error": "Nincs tétel a megadott rendeléshez!" }
```

---

---

## 1.2 Tételek keresése pizza név alapján

GET `/api/order-items/search?pizzaName=sonka`

### Leírás:

Listázza azokat a tételeket, amelyeknél a `rendeles` mező tartalmazza a megadott szöveget, **egységár szerint csökkenő**, azonos ár esetén **név szerint növekvő** sorrendben.

Sikeres válasz (200 OK):

```
[
  {
    "id": 4,
    "orderId": 2,
    "rendeles": "Sonkás",
    "mennyiség": 3,
    "egysegar": 2290
  }
]
```

### Hibakezelés:

- **400 Bad Request:** ha hiányzik a `pizzaName` query paraméter

```
{ "error": "A pizzaName megadása kötelező!" }
```

---

## 1.3 Drágább pizzatételek listázása minimum egységár alapján

GET `/api/order-items/price-over/{price}`

### Leírás:

Adja vissza azokat a tételeket, amelyek `egysegar` értéke nagyobb a megadott árnál, **egységár szerint csökkenő** sorrendben.

Példa: `/api/order-items/price-over/2800`

Sikeres válasz (200 OK):

```
[
  {
    "id": 15,
    "orderId": 5,
    "rendeles": "Carbonara",
    "menyiseg": 4,
    "egysegar": 3090
  }
]
```

Hibakezelés:

- 400 Bad Request: ha a megadott ár nem szám

```
{ "error": "Érvénytelen árparaméter!" }
```

---

## 2. Statisztikai feladatok

### 2.1 Egy rendelés teljes összegének kiszámítása

GET `/api/orders/{orderId}/total`

Leírás:

Számítsa ki a megadott rendelés teljes összegét a következő képlet alapján:

```
összeg = SUM(menyiseg * egysegar)
```

Sikeres válasz (200 OK):

```
{
  "orderId": 4,
  "totalAmount": 27690
}
```

Hibakezelés:

- 404 Not Found: ha a rendelés nem létezik

```
{ "error": "A megadott rendelés nem található!" }
```

---

---

## 2.2 Átlagos tételérték meghatározása

GET `/api/stats/average-line-value`

**Leírás:**

Számítsa ki az összes rendelési tétel átlagos értékét, ahol egy tétel értéke:

```
tételérték = mennyiség * egysegar
```

Az eredményt 1 tizedesre kerekítve adja vissza.

**Sikeres válasz (200 OK):**

```
{
  "averageLineValue": 5157.0
}
```

**Hibakezelés:**

- **404 Not Found:** ha nincs egyetlen tétel sem az adatbázisban

```
{ "error": "Nincs adat az adatbázisban!" }
```

---

## 3. CRUD feladatok

### 3.1 Új rendelési tétel rögzítése

POST `/api/order-items`

**Bemenet:**

```
{
  "orderId": 23,
  "rendeles": "Pepperoni",
  "mennyiség": 2,
  "egysegar": 2490
}
```

#### Leírás:

Rögzítsen új rendelési tételt.

#### Sikeres válasz (201 Created):

```
{
  "message": "Rendelési tétel sikeresen rögzítve!",
  "id": 61
}
```

#### Hibakezelés:

- **400 Bad Request:** ha bármelyik mező hiányzik vagy üres

```
{ "error": "Minden mező kitöltése kötelező!" }
```

---

## 3.2 Tétel mennyiségének módosítása

**PATCH** /api/order-items/{id}/quantity

#### Bemenet:

```
{
  "mennyiseg": 3
}
```

#### Leírás:

Módosítsa a megadott azonosítójú rendelési tétel mennyiségét.

#### Sikeres válasz (200 OK):

```
{ "message": "A mennyiség módosítása sikeres!" }
```

#### Hibakezelés:

- **404 Not Found:** ha az `id` nem létezik

```
{ "error": "Nincs ilyen azonosítójú tétel!" }
```

- 400 Bad Request: ha hiányzik a `menyiseg`, vagy nem pozitív egész szám

```
{ "error": "A mennyiseg mező kötelező és pozitív egész szám kell legyen!" }
```

---

### 3.3 Rendelési tétel törlése

DELETE `/api/order-items/{id}`

Leírás:

Törölje a megadott azonosítójú rendelési tételt.

Sikeres válasz (200 OK):

```
{ "message": "A rendelési tétel törlése sikeres!" }
```

Hibakezelés:

- 404 Not Found: ha az `id` nem létezik

```
{ "error": "Nincs ilyen azonosítójú tétel!" }
```

---

## 4. Csoportosításos feladatok

### 4.1 Rendelések összértéke rendelésenként

GET `/api/reports/orders/totals`

Leírás:

Csoportosítsa a tételeket `orderId` szerint, és számítsa ki minden rendelés teljes összegét.

Elvárt logika:

- csoportosítás `orderId` alapján
- összegzés: `SUM(mennyiseg * egysegar)`
- rendezés: teljes összeg szerint csökkenő sorrend

Sikeres válasz (200 OK):

```
[
  {
    "orderId": 4,
    "totalAmount": 27690
  },
  {
    "orderId": 5,
    "totalAmount": 26410
  }
]
```

---

## 4.2 Eladott darabszám pizzánként

GET `/api/reports/pizzas/quantities`

### Leírás:

Csoportosítsa a tételeket pizza név szerint, és számítsa ki, hogy az egyes pizzákból összesen hány darabot rendeltek.

### Elvárt logika:

- csoportosítás `rendeles` alapján
- összegzés: `SUM(mennyiseg)`
- rendezés: darabszám szerint csökkenő sorrend

### Sikeres válasz (200 OK):

```
[
  {
    "rendeles": "Sonkás",
    "totalQuantity": 19
  },
  {
    "rendeles": "Tonhalas",
    "totalQuantity": 14
  }
]
```

---

## 4.3 Árbevétel pizzánként

GET `/api/reports/pizzas/revenue`



### Leírás:

Csoportosítsa a tételeket pizza név szerint, és számítsa ki az egyes pizzák teljes árbevételét.

### Elvárt logika:

- csoportosítás `rendeles` alapján
- összegzés: `SUM(mennyiség * egysegar)`
- rendezés: árbevétel szerint csökkenő sorrend

### Sikeres válasz (200 OK):

```
[
  {
    "rendeles": "Sonkás",
    "revenue": 43510
  },
  {
    "rendeles": "Tonhalas",
    "revenue": 40460
  }
]
```

---

## Technikai elvárások

- Az adatbázis-kapcsolat legyen külön rétegben kezelve.
- A végpontok minden esetben JSON választ adjanak vissza.
- A hibakezeléshez használjon megfelelő HTTP státuszkódokat.
- A számítások az adatbázis alapján történjenek, ne fixen kódolt adatokkal.
- A megoldás során figyeljen a tiszta projektstruktúrára.

---

## Összesítés

A feladatsor tartalmaz:

- 3 db **SELECT / WHERE / ORDER BY** feladatot
- 2 db **statisztikai** feladatot
- 3 db **CRUD** feladatot
- 3 db **csoportosításos** feladatot

Összesen: 11 backend feladat

---

## Melléklet

---

- CSV adatfájl: `pizza_rendeles_tetelek.csv`